

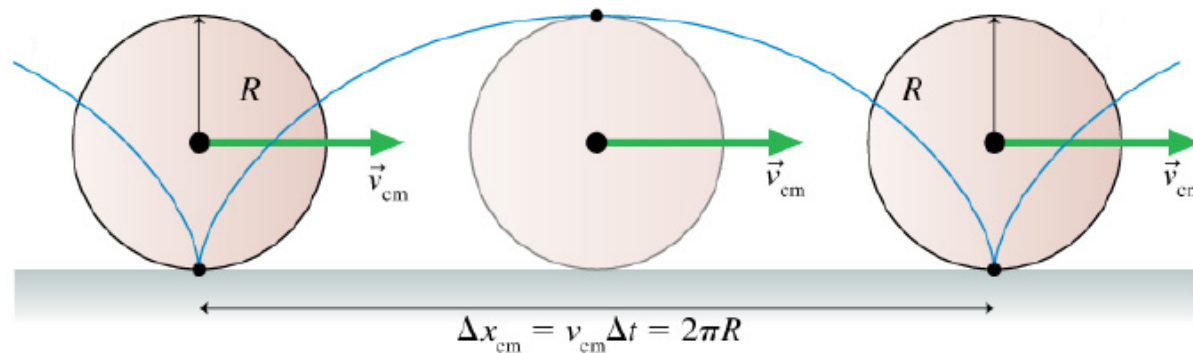
Rolling Motion

© James Wadsley, 2VG3

Pure Rolling Motion

1st Year Physics

- In pure rolling motion, an object rolls without slipping
- In such a case, there is a simple relationship between its rotational and translational motions
- **A combination of motion:** an object rotates about an axis that is moving along a straight line



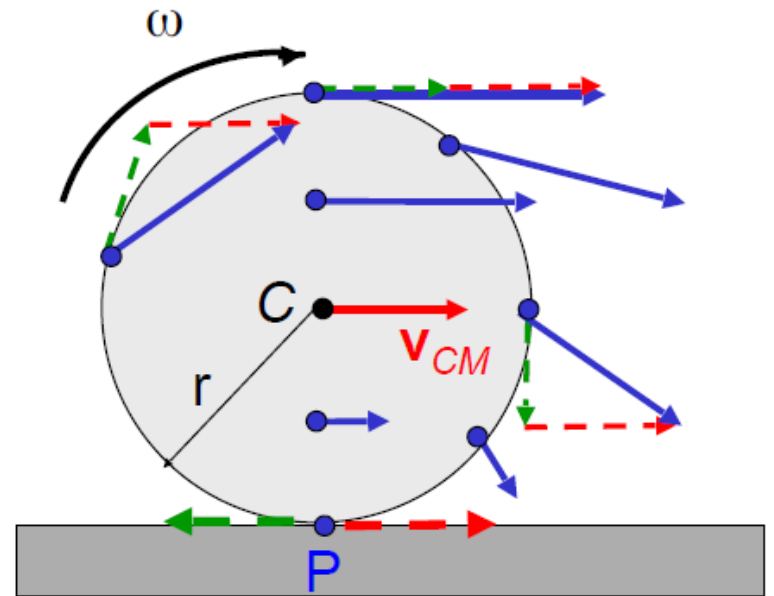
$$v_{CM} = \frac{2\pi R}{T} = R\omega$$

Rolling Motion

Different points on a rolling object have different velocities (blue vectors).

The **velocity of each point (blue)** is the **velocity of the centre (red)** plus the **velocity $\omega \times \mathbf{r}$ of that point relative to the centre (green)**:

$$\mathbf{v} = \mathbf{v}_{CM} + \omega \times \mathbf{r}$$



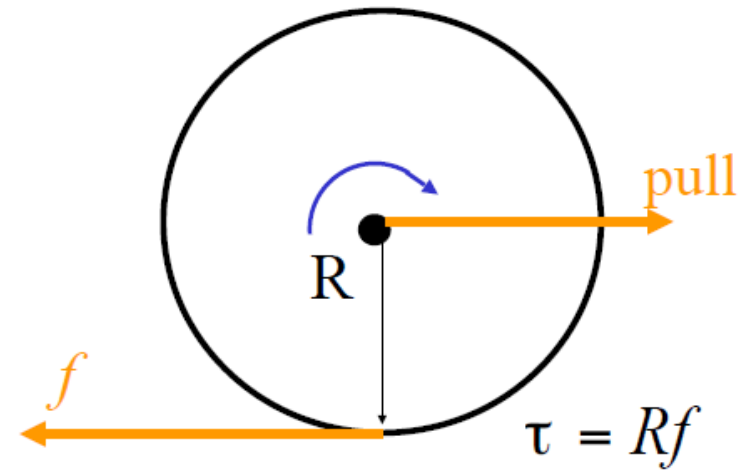
The point in contact with the ground, P , is momentarily stationary.

The point on the top of the wheel moves forward at twice the speed of the centre.

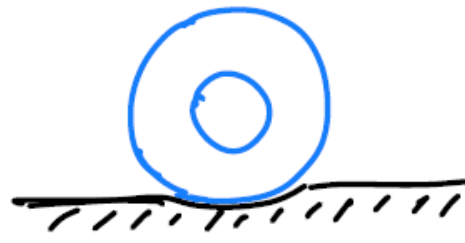
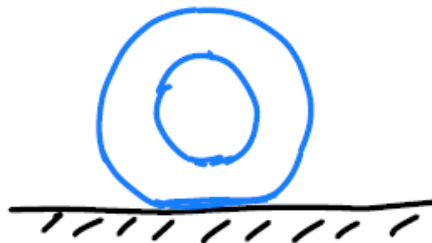
Friction and Rolling Resistance

*Friction is necessary to create a **torque** which can cause rotation and rolling.*

*This is **static friction** if the wheel rolls **without** slipping. At the point of contact between the road and wheel, there is no relative motion, and **friction does no work**.*



“**Rolling resistance**” which slows the wheel results from the tire (and the road surface) flexing inelastically at the contact point. This can be a small effect, so wheels roll a long way.



Implementing Rolling

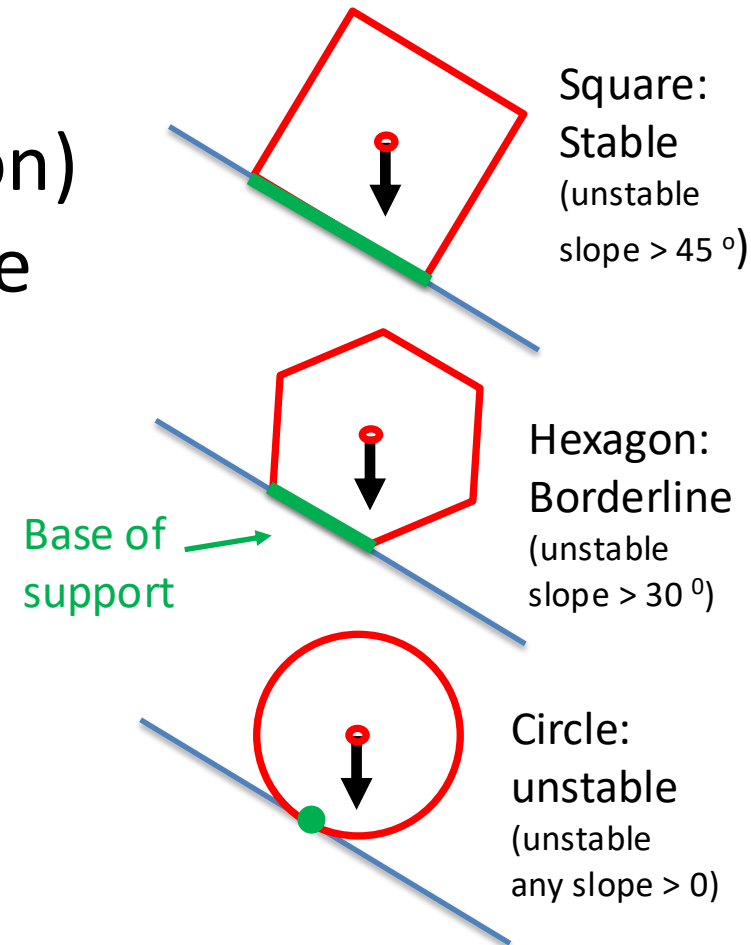
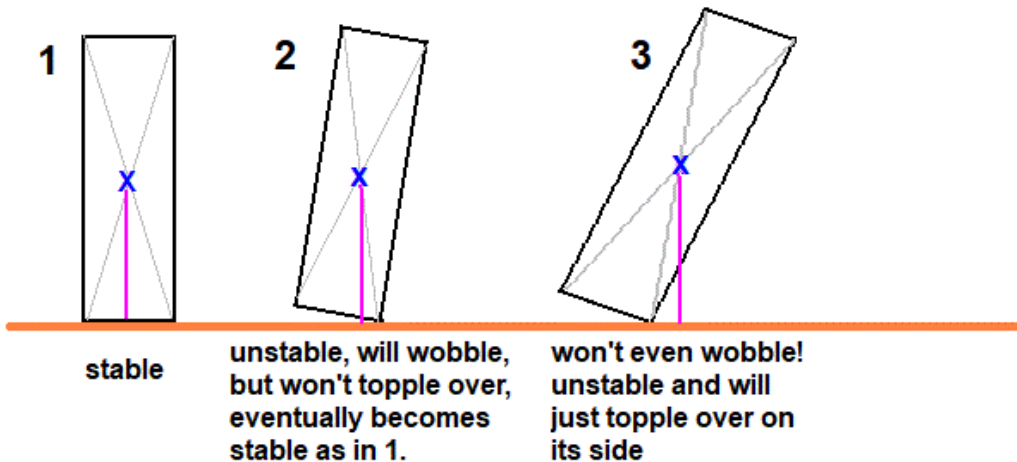
Circles vs. Polygons

- A circle can be considered to be a polygon with (infinitely) many sides
- If we increase the number of sides in our 2d rigid bodies from 4 (squares) to larger values, we should be able to model a cylinder
- Once we have more sides, the physics we already have should be sufficient to handle rolling objects

Circles vs. Polygons vs. Squares

- A circle can be considered to be a polygon with many sides.
- Tipping over => Rolling
Circle (or many-sided polygon)
always unstable on an incline

Centre of mass and stability



Steps to allow for added shapes

- collidePointPolygon already very general -- can handle any shape

Need:

- **Shape variable to indicate rigidbody shape**
- **Models for new shapes**
- Code to generate vertices based on shape
- Code to collide together generic shapes (replace collideSquareSquare with collideShapes)

Implementation

- Start from working copy of
rigidbody2d_friction.py
- Copy to:
rigidbody2d_rolling.py
- We will also update
rigidbody2d_collision_mod.py

Enable shape as property

edit rigidbody2d_mod.py

- We can just use a word to describe the shape
- e.g. A shape string like "square" or "cylinder" or various polygons, e.g. "nonagon"

```
class RigidBody2D(Particle):
    def __init__(self, *args, **kwargs):
        Particle.__init__(self, *args, **kwargs)
        self.theta = 0.0
        self.omega = 0.0
        self.inverseMomentOfInertia = self.inverseMass/(2./3.*self.radius**2)
        self.shape = "square"    #default shape
```

Enable shape as property

edit rigidbody2d_rolling.py

- Alice.org has a "Tube.egg" shape we can use
- It is actually a nonagon (9 sided). It is a little off centre so I had to tweak it. I also rotated it to be side on for our standard camera view

```
self.tube = loader.loadModel("Tube")
self.tube.setScale(1.3/1.41421356237)
self.tube.setHpr(90.,90.,0.)
self.tube.setPos(0,0.1,0)
```

```
self.RigidBody2Ds[0].radius = 1.0
self.RigidBody2Ds[0].collisionRadius = 1.8*self.RigidBody2Ds[0].radius
self.RigidBody2Ds[0].setScale(1.0)
self.RigidBody2Ds[0].setPos(-2.,3,self.RigidBody2Ds[0].radius-0.7)
self.RigidBody2Ds[0].vel = Vec3(-0.1,0,-0.9)
self.RigidBody2Ds[0].doGravity = True
self.RigidBody2Ds[0].omega = 0
self.RigidBody2Ds[0].shape = "cylinder"
self.tube.instanceTo(self.RigidBody2Ds[0])
self.RigidBody2Ds[0].inverseMomentOfInertia =
    self.RigidBody2Ds[0].inverseMass/(self.RigidBody2Ds[0].radius**2)
```

Note:
must disable default models now!

Cylinder (Nonagon) on an inclined plane

```
self.tube = loader.loadModel("Tube")
self.tube.setScale(1.3/1.41421356237)
self.tube.setHpr(90.,90.,0.)
self.tube.setPos(0,0.1,0)
```

```
self.RigidBody2Ds[0].radius = 1.0
self.RigidBody2Ds[0].collisionRadius = 1.8*self.RigidBody2Ds[0].radius
self.RigidBody2Ds[0].setScale(1.0)
self.RigidBody2Ds[0].setPos(-2.,3,self.RigidBody2Ds[0].radius-0.7)
self.RigidBody2Ds[0].vel = Vec3(-0.1,0,-0.9)
self.RigidBody2Ds[0].doGravity = True
self.RigidBody2Ds[0].omega = 0
self.RigidBody2Ds[0].shape = "nonagon"
self.tube.instanceTo(self.RigidBody2Ds[0])
self.RigidBody2Ds[0].inverseMomentOfInertia =
    self.RigidBody2Ds[0].inverseMass/(self.RigidBody2Ds[0].radius**2)
```

```
self.RigidBody2Ds[1].radius = 4.0
self.RigidBody2Ds[1].collisionRadius = 1.8*self.RigidBody2Ds[1].radius
self.RigidBody2Ds[1].setScale(4.0)
self.RigidBody2Ds[1].setPos(0.,3,-self.RigidBody2Ds[1].radius-1.)
self.RigidBody2Ds[1].vel = Vec3(0,0,0)
self.RigidBody2Ds[1].inverseMass = 0.0
self.RigidBody2Ds[1].inverseMomentOfInertia = 0.0
self.RigidBody2Ds[1].theta = 0.10016742116
self.cube.instanceTo(self.RigidBody2Ds[1])
```

Steps to allow for added shapes

- collidePointPolygon already very general -- can handle any shape

Need:

- Shape variable to indicate rigidbody shape
- Models for new shapes
- **Code to generate vertices based on shape**
- **Code to collide together generic shapes (replace collideSquareSquare with collideShapes)**

```

def getShapeVertices( scene, shape ):
    return vertices

def transformedShapeVertices( scene, shape, radius, x, angle ):
    return vertices

def collideShapes( scene, p1, p2 ):
    return contacts

```

Supercedes:

```

def collideSquareSquare( scene, x1, x2, radius1, radius2, angle1, angle2 ):

    # get vertices in model space
    square = [Vec3(-1.0,0.0,-1.0),Vec3(-1.0,0.0,1.0),Vec3(1.0,0.0,1.0),Vec3(1.0,0.0,-1.0)]

    # transform vertices to world space
    m1 = Mat3().rotateMatNormaxis( angle1*180/3.141592653589793238462, Vec3(0,1,0) )
    m2 = Mat3().rotateMatNormaxis( angle2*180/3.141592653589793238462, Vec3(0,1,0) )
    square1 = []
    square2 = []
    for i in range(4):
        square1.append(x1 + m1.xform(square[i]*radius1))
        square2.append(x2 + m2.xform(square[i]*radius2))

    # find contacts using collidePointPolygon
    contacts = []
    for i in range(4):
        dcl,xcl,normal = collidePointPolygon( scene, square1[i], square2, x1, x2 )
        if (xcl != None and (xcl-x1).length()<(square1[i]-x1).length()*1.01):
            markerAdd(scene, xcl,"collision")
            contacts.append( (dcl,xcl,-normal) )
        dcl,xcl,normal = collidePointPolygon( scene, square2[i], square1, x2, x1 )
        if (xcl != None and (xcl-x2).length()<(square2[i]-x2).length()*1.01):
            markerAdd(scene, xcl,"collision")
            contacts.append( (dcl,xcl,normal) )

    return contacts

```

```
def getShapeVertices( scene, shape ):
    return vertices
```

```
def transformedShapeVertices( scene, shape, radius, x, angle ):
    return vertices
```

```
def collideShapes( scene, p1, p2 )
    return contacts
```

Modified interface:
Send whole rigidbody, rather than
individual properties

Supercedes:

```
def collideSquareSquare( scene, x1, x2, radius1, radius2, angle1, angle2 ):
```

```
    # get vertices in model space
```

```
    square = [Vec3(-1.0,0.0,-1.0),Vec3(-1.0,0.0,1.0),Vec3(1.0,0.0,1.0),Vec3(1.0,0.0,-1.0)]
```

```
    # transform vertices to world space
```

```
    m1 = Mat3().rotateMatNormaxis( angle1*180/3.141592653589793238462, Vec3(0,1,0) )
```

```
    m2 = Mat3().rotateMatNormaxis( angle2*180/3.141592653589793238462, Vec3(0,1,0) )
```

```
    square1 = []
```

```
    square2 = []
```

```
    for i in range(4):
```

```
        square1.append(x1 + m1.xform(square[i]*radius1))
```

```
        square2.append(x2 + m2.xform(square[i]*radius2))
```

```
    # find contacts using collidePointPolygon
```

```
    contacts = []
```

```
    for i in range(4):
```

```
        dc1,xcl,normal = collidePointPolygon( scene, square1[i], square2, x1, x2 )
```

```
        if (xcl != None and (xcl-x1).length()<(square1[i]-x1).length()*1.01):
```

```
            markerAdd(scene, xcl,"collision")
```

```
            contacts.append( (dc1,xcl,-normal) )
```

```
        dc1,xcl,normal = collidePointPolygon( scene, square2[i], square1, x2, x1 )
```

```
        if (xcl != None and (xcl-x2).length()<(square2[i]-x2).length()*1.01):
```

```
            markerAdd(scene, xcl,"collision")
```

```
            contacts.append( (dc1,xcl,normal) )
```

```
    return contacts
```

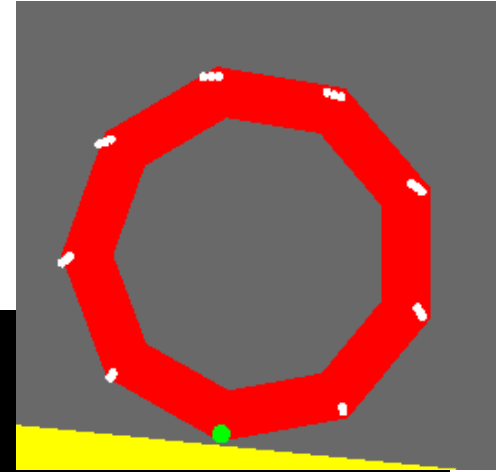
Square vs. Nonagon

rigidbody2d_collision_mod.py

- Tube cross-section is actually 9-sided -- can implement a nonagon too:

```
from math import *

def getShapeVertices( scene, shape ):
    if (shape == "square"):
        return [Vec3(-1.0,0.0,-1.0), Vec3(-1.0,0.0,1.0), Vec3(1.0,0.0,1.0),
Vec3(1.0,0.0,-1.0)]
    if (shape == "nonagon"):
        v = []
        for i in range(9):
            theta = i*2*pi/9.
            v.append( Vec3( cos(theta), 0., sin(theta) ) )
        return v
    if (shape == "cylinder"):
        v = []
        for i in range(360):
            theta = i*2*pi/360.
            v.append( Vec3( cos(theta), 0., sin(theta) ) )
        return v
```



Square vs. Nonagon

rigidbody2d_collision_mod.py

```
def transformedShapeVertices( scene, shape, radius, x, angle ):  
    #This should only be used for polygons!  
  
    m = Mat3().rotateMatNormaxis( angle*180/3.141592653589793238462,  
Vec3(0,1,0) )  
  
    vmodel = getShapeVertices(scene, shape)  
    vertices = []  
  
    for v in vmodel:  
        vertices.append(x + m.xform(v*radius))  
    return vertices
```

Square vs. Nonagon

rigidbody2d_collision_mod.py

```
def collideShapes( scene, p1, p2 ):

    x1 = p1.getPos()
    x2 = p2.getPos()
    v1 = transformedShapeVertices(scene, p1.shape, p1.radius, x1, p1.theta )
    v2 = transformedShapeVertices(scene, p2.shape, p2.radius, x2, p2.theta )

    for v in v1:
        markerAdd(scene, v, "corner" )

    contacts = []
    for v in v1:
        dc1,xc1,normal = collidePointPolygon( scene, v, v2, x1, x2 )
        if (xc1 != None and (xc1-x1).length()<(v-x1).length()*1.01):
            markerAdd(scene,xc1,"collision")
            contacts.append( (dc1,xc1,-normal) )

    for v in v2:
        dc1,xc1,normal = collidePointPolygon( scene, v, v1, x2, x1 )
        if (xc1 != None and (xc1-x2).length()<(v-x2).length()*1.01):
            markerAdd(scene, xc1,"collision")
            contacts.append( (dc1,xc1,normal) )

    return contacts
```

Code final touches

- Make sure to set up the objects correctly
 - set inverseMass, radius, collision radius, inverse moment of inertia correctly
 - Set shape and appropriate model
 - Make sure rolling object does not start inside the inclined plane (lower) object
- updateRigidBody2Ds:
 - Change call to collideSquareSquare to collideShapes!

Position correction & Rolling

- A rolling contact should not be moving relative the ground
- This means position corrections parallel to the ground not needed (only useful for sliding objects)

```
#add accumulated change to rigid bodies
#Divide total effect by number of contacts
if (ncontacts > 0):
    pi.vel += dpiVel/ncontacts
    pj.vel += dpjVel/ncontacts
    pi.omega += dpiOmega/ncontacts
    pj.omega += dpjOmega/ncontacts
    if (self.fixPos):
        pi.setPos(xi + nij*(pi.rold-xi).dot(nij) )# +dpiPos/ncontacts)
        pj.setPos(xj + nij*(pj.rold-xj).dot(nij) )# +dpjPos/ncontacts)
```

Testing/Exploring

Nonagon doesn't move!?



Testing:

Cylinder (Nonagon) on an inclined plane

```
self.tube = loader.loadModel("Tube")
self.tube.setScale(1.3/1.41421356237)
self.tube.setHpr(90.,90.,0.)
self.tube.setPos(0,0.1,0)
```

Option one – start it spinning!

```
self.RigidBody2Ds[0].radius = 1.0
self.RigidBody2Ds[0].collisionRadius = 1.8*self.RigidBody2Ds[0].radius
self.RigidBody2Ds[0].setScale(1.0)
self.RigidBody2Ds[0].setPos(-2.,3,self.RigidBody2Ds[0].radius-0.7)
self.RigidBody2Ds[0].vel = Vec3(-0.1,0,-0.9)
self.RigidBody2Ds[0].doGravity = True
self.RigidBody2Ds[0].omega = 3 # need to spin it to move it!
self.RigidBody2Ds[0].shape = "nonagon"
self.tube.instanceTo(self.RigidBody2Ds[0])
```

```
self.RigidBody2Ds[0].inverseMomentOfInertia =
self.RigidBody2Ds[0].inverseMass/(self.RigidBody2Ds[0].radius**2)
```

```
self.RigidBody2Ds[1].collisionRadius = 1.8*self.RigidBody2Ds[1].radius
self.RigidBody2Ds[1].setScale(4.0)
self.RigidBody2Ds[1].setPos(0.,3,-self.RigidBody2Ds[1].radius-1.)
self.RigidBody2Ds[1].vel = Vec3(0,0,0)
self.RigidBody2Ds[1].inverseMass = 0.0
self.RigidBody2Ds[1].inverseMomentOfInertia = 0.0
self.RigidBody2Ds[1].theta = 0.10016742116
```

Testing:

Cylinder (Nonagon) on an inclined plane

```
self.tube = loader.loadModel("Tube")
self.tube.setScale(1.3/1.41421356237)
self.tube.setHpr(90.,90.,0.)
self.tube.setPos(0,0.1,0)
```

Option two – tilt the plane more

```
self.RigidBody2Ds[0].radius = 1.0
self.RigidBody2Ds[0].collisionRadius = 1.8*self.RigidBody2Ds[0].radius
self.RigidBody2Ds[0].setScale(1.0)
self.RigidBody2Ds[0].setPos(-2.,3,self.RigidBody2Ds[0].radius-0.7 +1 )
self.RigidBody2Ds[0].vel = Vec3(-0.1,0,-0.9)
self.RigidBody2Ds[0].doGravity = True
self.RigidBody2Ds[0].omega = 0
self.RigidBody2Ds[0].shape = "cylinder"
self.tube.instanceTo(self.RigidBody2Ds[0])
```

```
self.RigidBody2Ds[0].inverseMomentOfInertia =
self.RigidBody2Ds[0].inverseMass/(self.RigidBody2Ds[0].radius**2)
```

```
self.RigidBody2Ds[1].collisionRadius = 1.8*self.RigidBody2Ds[1].radius
self.RigidBody2Ds[1].setScale(4.0)
self.RigidBody2Ds[1].setPos(0.,3,-self.RigidBody2Ds[1].radius-1.)
self.RigidBody2Ds[1].vel = Vec3(0,0,0)
self.RigidBody2Ds[1].inverseMass = 0.0
self.RigidBody2Ds[1].inverseMomentOfInertia = 0.0
self.RigidBody2Ds[1].theta = 0.3 # or make it steeper!
```


Testing:

Cylinder (Nonagon) on an inclined plane

```
self.tube = loader.loadModel("Tube")
self.tube.setScale(1.3/1.41421356237)
self.tube.setHpr(90.,90.,0.)
self.tube.setPos(0,0.1,0)
```

Option three – use a 360 sided
polygon “cylinder”

```
self.RigidBody2Ds[0].radius = 1.0
self.RigidBody2Ds[0].collisionRadius = 1.8*self.RigidBody2Ds[0].radius
self.RigidBody2Ds[0].setScale(1.0)
self.RigidBody2Ds[0].setPos(-2.,3,self.RigidBody2Ds[0].radius-0.7)
self.RigidBody2Ds[0].vel = Vec3(-0.1,0,-0.9)
self.RigidBody2Ds[0].doGravity = True
self.RigidBody2Ds[0].omega = 0
self.RigidBody2Ds[0].shape = "cylinder"
self.tube.instanceTo(self.RigidBody2Ds[0])
```

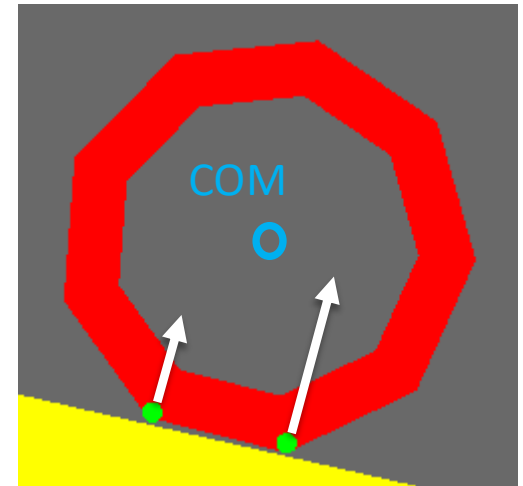
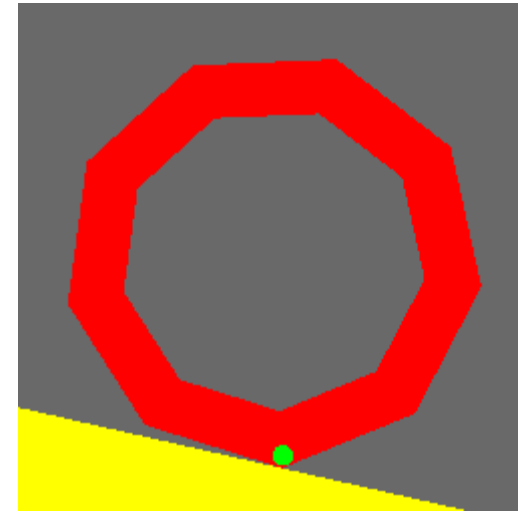
```
self.RigidBody2Ds[0].inverseMomentOfInertia =
self.RigidBody2Ds[0].inverseMass/(self.RigidBody2Ds[0].radius**2)
```

```
self.RigidBody2Ds[1].collisionRadius = 1.8*self.RigidBody2Ds[1].radius
self.RigidBody2Ds[1].setScale(4.0)
self.RigidBody2Ds[1].setPos(0.,3,-self.RigidBody2Ds[1].radius-1.)
self.RigidBody2Ds[1].vel = Vec3(0,0,0)
self.RigidBody2Ds[1].inverseMass = 0.0
self.RigidBody2Ds[1].inverseMomentOfInertia = 0.0
self.RigidBody2Ds[1].theta = 0.10016742116
```

Rolling & Friction Cylinders

Cylinders vs. Polygons

- A polygon typically has one point touching the inclined plane at all times (and sometimes two)
- The next point in sequence will slam into the plane which dissipates energy (for $e < 1$)
- It will have a large **impulse** and a net torque opposing the rolling motion (lose rotational energy)
- Polygons are not great cylinders even with many sides



Coding along...

- We initially started with:
`rigidbody2d_friction.py`
- We implement general shapes into:
`rigidbody2d_mod.py`
`rigidbody2d_collision_mod.py`

```
def getShapeVertices( scene, shape ):  
    return vertices
```

```
def transformedShapeVertices( scene, shape, radius, x, angle ):  
    # get model space vertices <-- use getShapeVertices  
    return vertices
```

```
def collideShapes( scene, p1, p2 ):  
    # get transformed vertices <-- use transformedShapeVertices  
  
    # find contacts using collidePointPolygon  
    return contacts
```

Replace:

```
def collideSquareSquare( scene, x1, x2, radius1, radius2, angle1, angle2 ):  
  
    # get vertices in model space  
    square = [Vec3(-1.0,0.0,-1.0),Vec3(-1.0,0.0,1.0),Vec3(1.0,0.0,1.0),Vec3(1.0,0.0,-1.0)]  
  
    # transform vertices to world space  
    m1 = Mat3().rotateMatNormaxis( angle1*180/3.141592653589793238462, Vec3(0,1,0) )  
    m2 = Mat3().rotateMatNormaxis( angle2*180/3.141592653589793238462, Vec3(0,1,0) )  
    square1 = []  
    square2 = []  
    for i in range(4):  
        square1.append(x1 + m1.xform(square[i]*radius1))  
        square2.append(x2 + m2.xform(square[i]*radius2))  
  
    # find contacts using collidePointPolygon  
    contacts = []  
    for i in range(4):  
        dc1,xcl,normal = collidePointPolygon( scene, square1[i], square2, x1, x2 )  
        if (xcl != None and (xcl-x1).length()<(square1[i]-x1).length()*1.01):  
            markerAdd(scene, xcl,"collision")  
            contacts.append( (dc1,xcl,-normal) )  
        dc1,xcl,normal = collidePointPolygon( scene, square2[i], square1, x2, x1 )  
        if (xcl != None and (xcl-x2).length()<(square2[i]-x2).length()*1.01):  
            markerAdd(scene, xcl,"collision")  
            contacts.append( (dc1,xcl,normal) )  
  
    return contacts
```

Previously...

```

def getShapeVertices( scene, shape ):
    return vertices

def transformedShapeVertices( scene, shape, radius, x, angle ):
    # get model space vertices <-- use getShapeVertices
    return vertices

def collideShapes( scene, p1, p2 ):
    # get transformed vertices <-- use transformedShapeVertices

    # find contacts using collidePointPolygon
    return contacts

```

Previously...

Replace:

```

def collideSquareSquare( scene, x1, x2, radius1, radius2, angle1, angle2 ):

    # get vertices in model space
    square = [Vec3(-1.0,0.0,-1.0),Vec3(-1.0,0.0,1.0),Vec3(1.0,0.0,1.0),Vec3(1.0,0.0,-1.0)]

    # transform vertices to world space
    m1 = Mat3().rotateMatNormaxis( angle1*180/3.141592653589793238462, Vec3(0,1,0) )
    m2 = Mat3().rotateMatNormaxis( angle2*180/3.141592653589793238462, Vec3(0,1,0) )
    square1 = []
    square2 = []
    for i in range(4):
        square1.append(x1 + m1.xform(square[i]*radius1))
        square2.append(x2 + m2.xform(square[i]*radius2))

    # find contacts using collidePointPolygon
    contacts = []
    for i in range(4):
        dc1,xcl,normal = collidePointPolygon( scene, square1[i], square2, x1, x2 )
        if (xcl != None and (xcl-x1).length()<(square1[i]-x1).length()*1.01):
            markerAdd(scene, xcl,"collision")
            contacts.append( (dc1,xcl,-normal) )
        dc1,xcl,normal = collidePointPolygon( scene, square2[i], square1, x2, x1 )
        if (xcl != None and (xcl-x2).length()<(square2[i]-x2).length()*1.01):
            markerAdd(scene, xcl,"collision")
            contacts.append( (dc1,xcl,normal) )

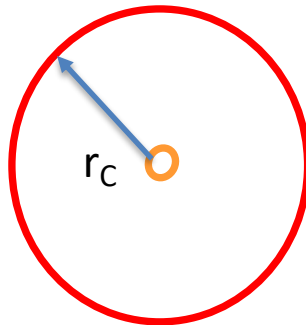
    return contacts

```

With cylinders we will no longer need to use the yellow code for the race part of the homework

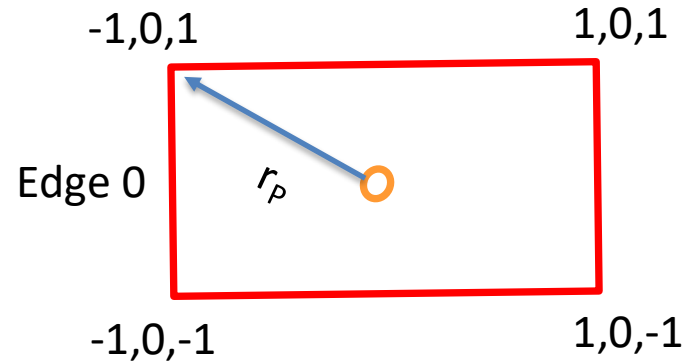
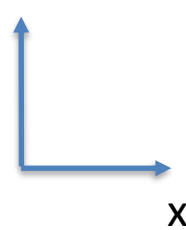
True Cylinders

- Cylinder-polygon collisions are easier than polygon-polygon collisions
- Cylinders (2D) = circle = disk only have a centre and a radius (as per old sphere-sphere collisions)



Position of Polygon Vertices: same as before

Consider a list of offsets r_p from the centre of mass of a rigid body to locate all the vertices

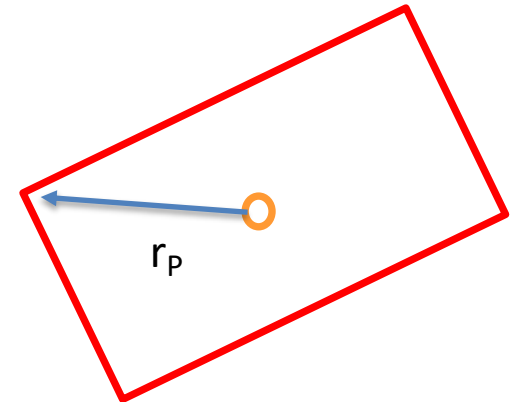


In **model space** for a cube of radius r
(side $s = 2r$)

4 corners (assume x,z plane as in Panda3d)

$$r_p = (-1,0,-1), (-1,0,1), (1,0,1), (1,0,-1)$$

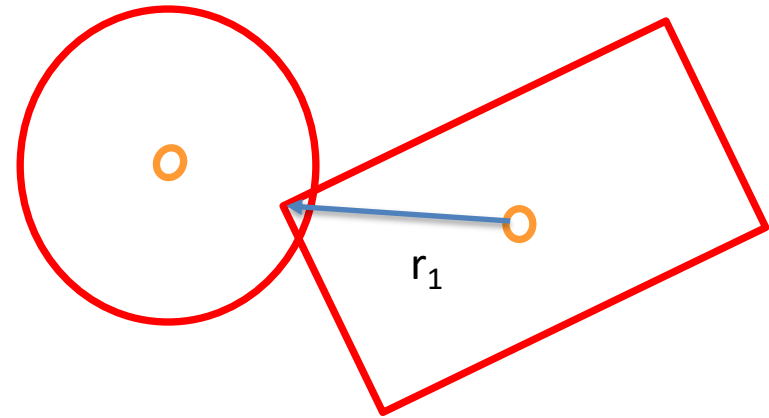
We can use a rotation matrix to get a **world space equivalent**



Cylinder - Polygon collision

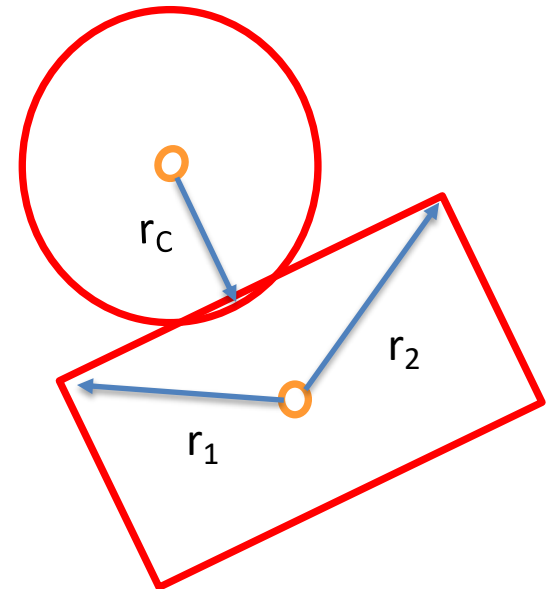
Case 1: Corner of polygon pokes into cylinder: similar to sphere-sphere collision

Normal set by cylinder radial direction



Case 2: Cylinder pokes into edge: Can use `collidePointLine`

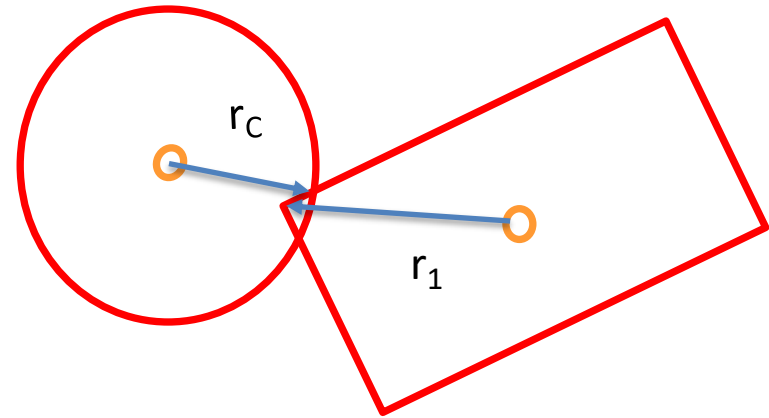
Normal set by edge normal



Cylinder - Polygon collision

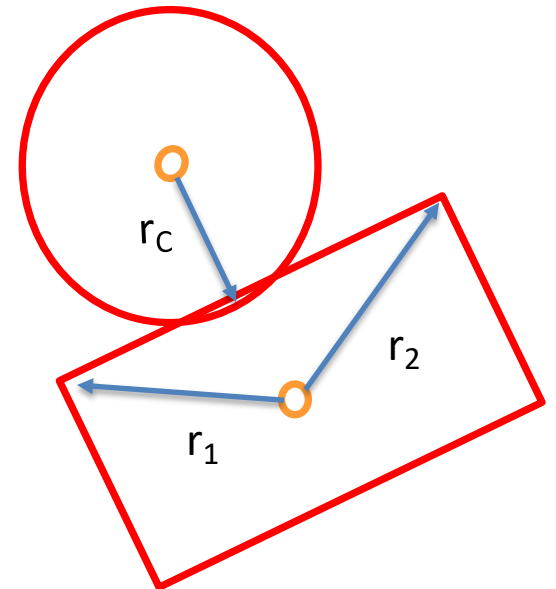
Case 1: Corner of polygon pokes into cylinder: similar to sphere-sphere collision

Normal set by cylinder radial direction



Case 2: Cylinder pokes into edge: Can use `collidePointLine`

Normal set by edge normal



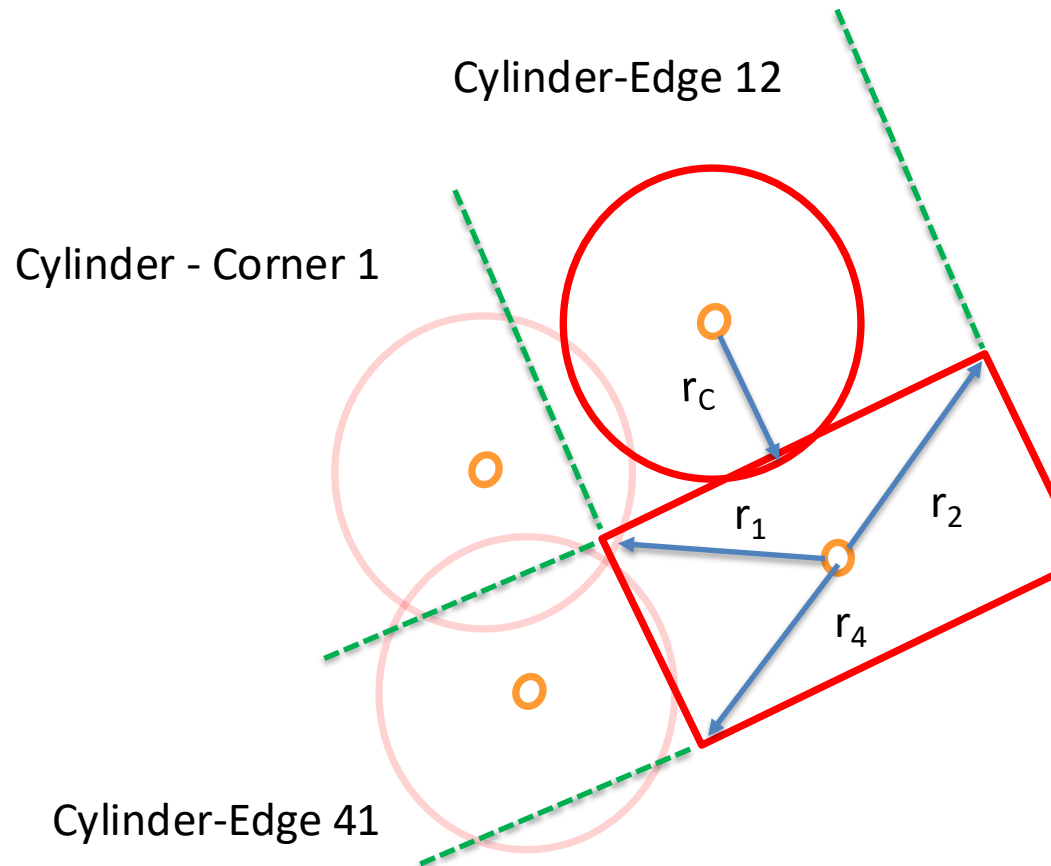
Cylinder - Polygon collision

Note lack of ambiguity:

- Can only get one case at a time for small penetration depths.
- Depends on where centre of mass of cylinder (circle) is.

Will just have one contact to deal with at most!

Except maybe round off or similar errors ...



Cylinder - Polygon

- If just one of the shapes is a cylinder we can short circuit `collideShapes` to handle that case

```
def collideShapes( scene, p1, p2 ):  
    if (p1.shape == "cylinder"):  
        return scene.collideCylinderPolygon( p1, p2, -1 )  
    elif (p2.shape == "cylinder"):  
        return scene.collideCylinderPolygon( p2, p1, 1 )  
  
    #old code for two polygonal shapes ...
```

collideCylinderPolygon

```
def collideCylinderPolygon( scene, pCyl, pPoly, sign ):
    #use transformedShapeVertices to get vertices for polygon

    nvertices = len(vertices)
    for i in range(nvertices):
        x1 = vertices[i]
        x2 = vertices[(i+1)%nvertices]

        # Test if cylinder is in edge with collidePointLine
        # Estimate depth, normal etc... and save contact if valid

    for i in range(nvertices):
        xv = vertices[i]

        # See if corner is poking into cylinder

    return contacts
```

collideCylinderPolygon

```
def collideCylinderPolygon( scene, pCyl, pPoly, sign ):
    #use transformedShapeVertices to get vertices for polygon

    nvertices = len(vertices)
    for i in range(nvertices):
        x1 = vertices[i]
        x2 = vertices[(i+1)%nvertices]

        # Test if cylinder is in edge with collidePointLine
        # Estimate depth, normal etc... and save contact if valid

    for i in range(nvertices):
        xv = vertices[i]

        # See if corner is poking into cylinder

    return contacts
```

Note: should be careful
about cases where
number of contacts > 1

Cylinder - Polygon collision

Contact details: depth, x_{contact} , normal

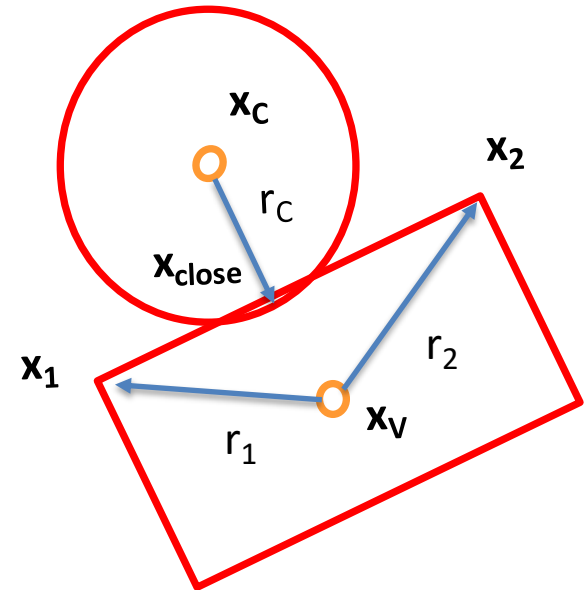
Case 1: Cylinder pokes into edge: use
collidePointLine

x_{close} from collidePointLine with (x_c, x_1, x_2)

depth = $r_c - |x_{\text{corner}} - x_c|$

Normal set by edge normal as before

$n = \frac{1}{2} (x_1 + x_2) - x_v$ normalized



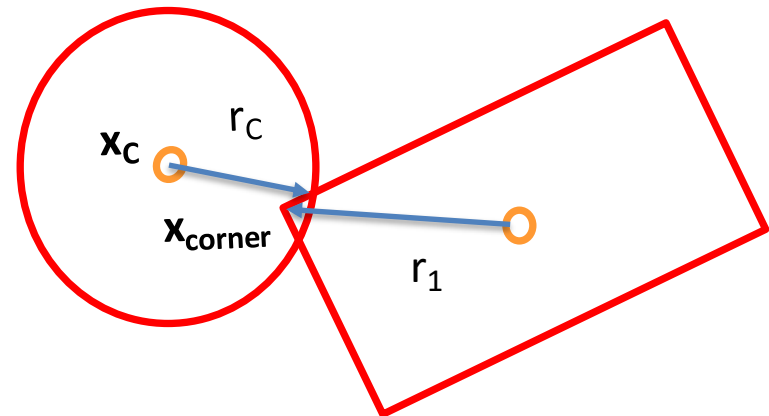
Case 2: Corner of polygon pokes into
cylinder

$x_{\text{contact}} = x_{\text{corner}}$ from shape

depth = $r_c - |x_{\text{corner}} - x_c|$

Normal set by cylinder radial direction:

$n = x_{\text{corner}} - x_c$ normalized



Cylinder - Polygon collision

Contact details: depth, x_{contact} , normal

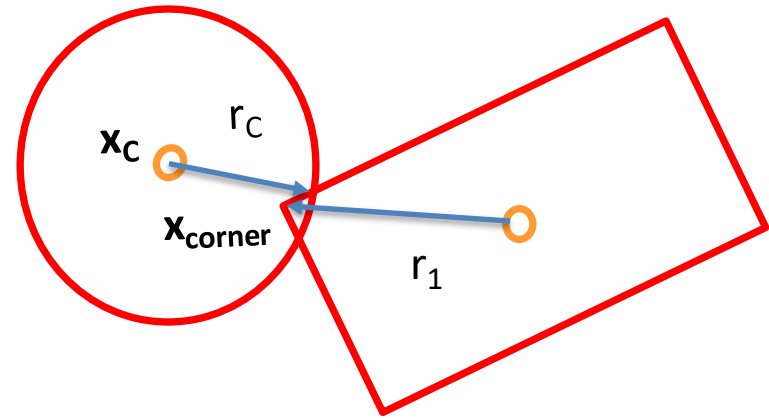
Case 1: Corner of polygon pokes into cylinder

$x_{\text{contact}} = x_{\text{corner}}$ from shape

depth = $r_c - |x_{\text{corner}} - x_c|$

Normal set by cylinder radial direction:

$n = x_{\text{corner}} - x_c$ normalized



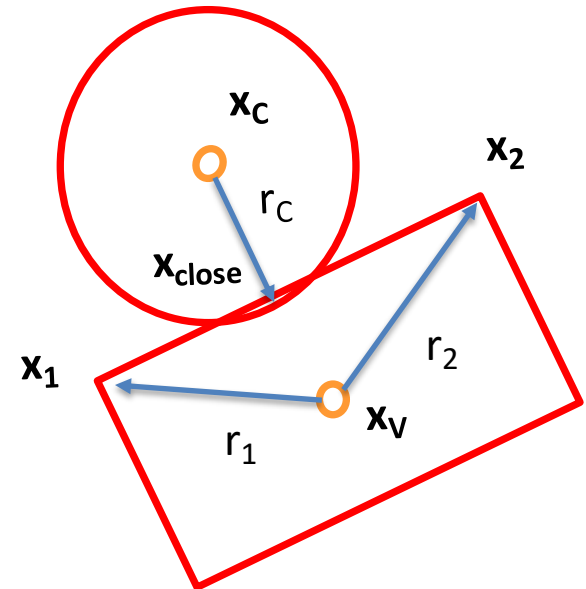
Case 2: Cylinder pokes into edge: use collidePointLine

x_{close} from collidePointLine with (x_c, x_1, x_2)

depth = $r_c - |x_{\text{close}} - x_c|$

Normal set by edge normal as before

$n = \frac{1}{2} (x_1 + x_2) - x_v$ normalized



collideCylinderPolygon

```
collideCylinderPolygon( scene, pCyl, p2, sign ):  
    contacts = []  
    xcenc = pCyl.getPos()  
    xcenv = p2.getPos()  
    vertices = transformedShapeVertices(scene, p2.shape, p2.radius, xcenv, p2.theta )  
    nvertices = len(vertices)  
  
    xcontacts = []  
    # Case 1: Test if cylinder is in eacg edge with collidePointLine  
    for i in range(nvertices):  
        x1 = vertices[i]  
        x2 = vertices[(i+1)%nvertices]  
        xcontact = collidePointLine( scene, xcenc, x1, x2 )  
        xcontacts.append(xcontact)  
        if (xcontact == None): continue  
        if ((xcontact-xcenc).length() > pCyl.radius): continue  
        markerAdd(scene,xcenc,"corner")  
        normal = ((x1+x2)*0.5-xcenv).normalized()  
        depth = pCyl.radius+(xcontact-xcenc).dot(normal)  
        if (depth >= 0):  
            markerAdd(scene,xcontact,"collision")  
            contacts.append( (depth, xcontact, normal*sign) )  
  
    # Case 2: See if corners are poking into cylinder  
    for i in range(nvertices):  
        xv = vertices[i]  
        if (xcontacts[i]!=None or xcontacts[(i+nvertices-1)%nvertices]!=None): continue  
        rcc = xv-xcenc  
        if (rcc.length() > pCyl.radius): continue  
        normal = rcc.normalized()  
        depth = pCyl.radius-rcc.length()  
        markerAdd(scene,xv,"collision")  
        contacts.append( (depth, xv, normal) )  
  
    assert( len(contacts) <= 1)  
    return contacts
```

Collide: Cylinder with Polygon

rigidbody2d_collision_mod.py

```
def collideShapes( scene, p1, p2 ):
    if (p1.shape == "cylinder"):
        return collideCylinderPolygon( scene, p1, p2, -1 )
    elif (p2.shape == "cylinder"):
        return collideCylinderPolygon( scene, p2, p1, 1 )

    x1 = p1.getPos()
    x2 = p2.getPos()
    v1 = transformedShapeVertices(scene, p1.shape, p1.radius, x1, p1.theta )
    v2 = transformedShapeVertices(scene, p2.shape, p2.radius, x2, p2.theta )

    for v in v1:
        markerAdd(scene, v, "corner" )

    contacts = []
    for v in v1:
        dc1,xc1,normal = collidePointPolygon( scene, v, v2, x1, x2 )
        if (xc1 != None and (xc1-x1).length()<(v-x1).length()*1.01):
            markerAdd(scene,xc1,"collision")
            contacts.append( (dc1,xc1,-normal) )

    for v in v2:
        dc1,xc1,normal = collidePointPolygon( scene, v, v1, x2, x1 )
        if (xc1 != None and (xc1-x2).length()<(v-x2).length()*1.01):
            markerAdd(scene, xc1,"collision")
            contacts.append( (dc1,xc1,normal) )

    return contacts
```

Testing:

True cylinder on an inclined plane

```
self.tube = loader.loadModel("Tube")
self.tube.setScale(1.3/1.41421356237)
self.tube.setHpr(90.,90.,0.)
self.tube.setPos(0,0.1,0)

self.RigidBody2Ds[0].radius = 1.0
self.RigidBody2Ds[0].collisionRadius = 1.8*self.RigidBody2Ds[0].radius
self.RigidBody2Ds[0].setScale(1.0)
self.RigidBody2Ds[0].setPos(-2.,3,self.RigidBody2Ds[0].radius-0.7)
self.RigidBody2Ds[0].vel = Vec3(-0.1,0,-0.9)
self.RigidBody2Ds[0].doGravity = True
self.RigidBody2Ds[0].omega = 0 # don't need to spin it to move it!
self.RigidBody2Ds[0].shape = "cylinder"
self.tube.instanceTo(self.RigidBody2Ds[0])
self.RigidBody2Ds[0].inverseMomentOfInertia =
self.RigidBody2Ds[0].inverseMass/(self.RigidBody2Ds[0].radius**2)

self.RigidBody2Ds[1].collisionRadius = 1.8*self.RigidBody2Ds[1].radius
self.RigidBody2Ds[1].setScale(4.0)
self.RigidBody2Ds[1].setPos(0.,3,-self.RigidBody2Ds[1].radius-1.)
self.RigidBody2Ds[1].vel = Vec3(0,0,0)
self.RigidBody2Ds[1].inverseMass = 0.0
self.RigidBody2Ds[1].inverseMomentOfInertia = 0.0
self.RigidBody2Ds[1].theta = 0.10016742116
```

Rolling & Friction Expectations

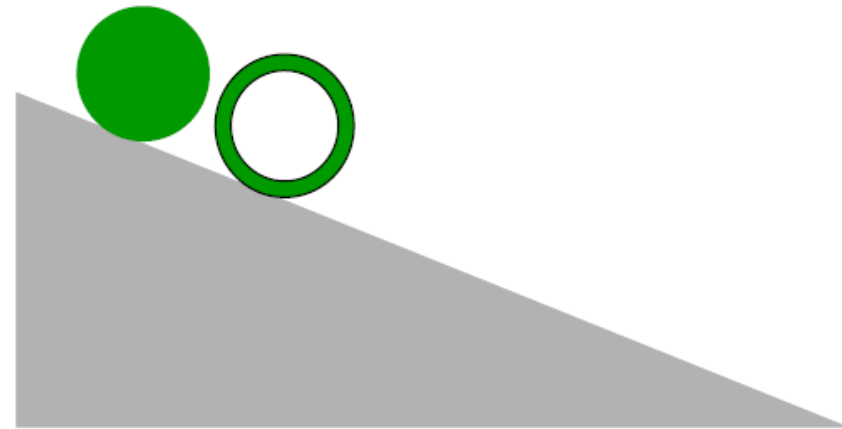
Testing

Quiz:

A solid cylinder and a hollow pipe roll down a ramp without slipping.

Which one move with larger acceleration?

- a) the cylinder
- b) the pipe
- c) they move with the same acceleration

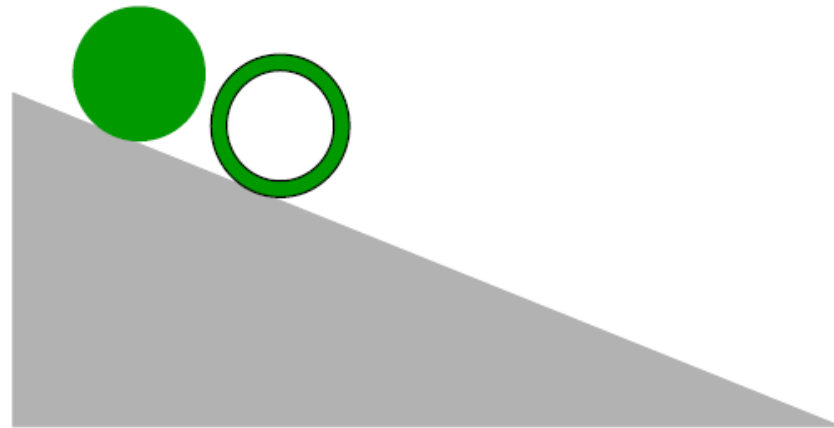


Testing

Quiz:

A solid cylinder and a hollow pipe roll down a ramp without slipping.

Which one move with larger acceleration?



- ✓ a) the cylinder
- b) the pipe
- c) they move with the same acceleration

Different Moments of Inertia

$$U_i = mgh$$

$$\begin{aligned} K_f &= \frac{1}{2}mv^2 + \frac{1}{2}I\omega^2 \\ &= \frac{1}{2}mv^2 + \frac{1}{2}I\left(\frac{v}{R}\right)^2 \\ &= \frac{1}{2}\left(m + \frac{I}{R^2}\right)v^2 \end{aligned}$$

$$U_i = K_f$$

$$mgh = \frac{1}{2}\left(m + \frac{I}{R^2}\right)v^2 \quad \Rightarrow \quad v = \sqrt{\frac{2gh}{1 + I/(mR^2)}}$$



$$I_{\text{Solid Cylinder}} = \frac{1}{2}mR^2$$

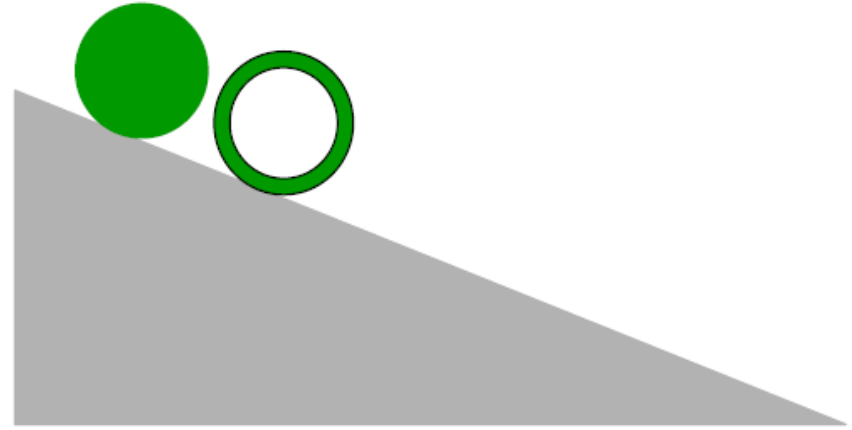
$$I_{\text{Pipe}} = mR^2$$

$$I_{\text{general}} = c(mR^2)$$

$$v = \sqrt{\frac{2gh}{1 + c}}$$

Exploration

A solid cylinder and a hollow pipe roll down a ramp without slipping.



- a) Which gets to the bottom first?
- b) If they have equal masses, which has the greatest kinetic energy at the bottom?
- c) If the surface were slippery, would the time increase or decrease?
- d) For a moderate coefficient of friction, which can roll down the steepest slope without slipping?

The Race

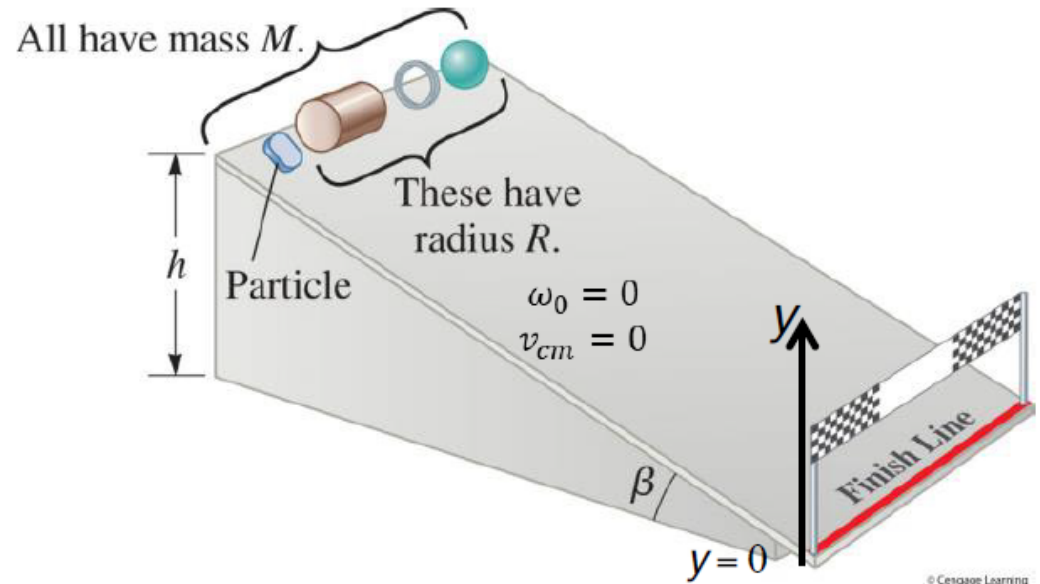
Downhill race between four objects

Hoop: $I = MR^2$

Cylinder: $I = \frac{1}{2}MR^2$

Sphere: $I = \frac{2}{5}MR^2$

Skier (particle)



The winner is:

- A. Hoop
- B. Cylinder
- C. Sphere
- D. Skier

The Race

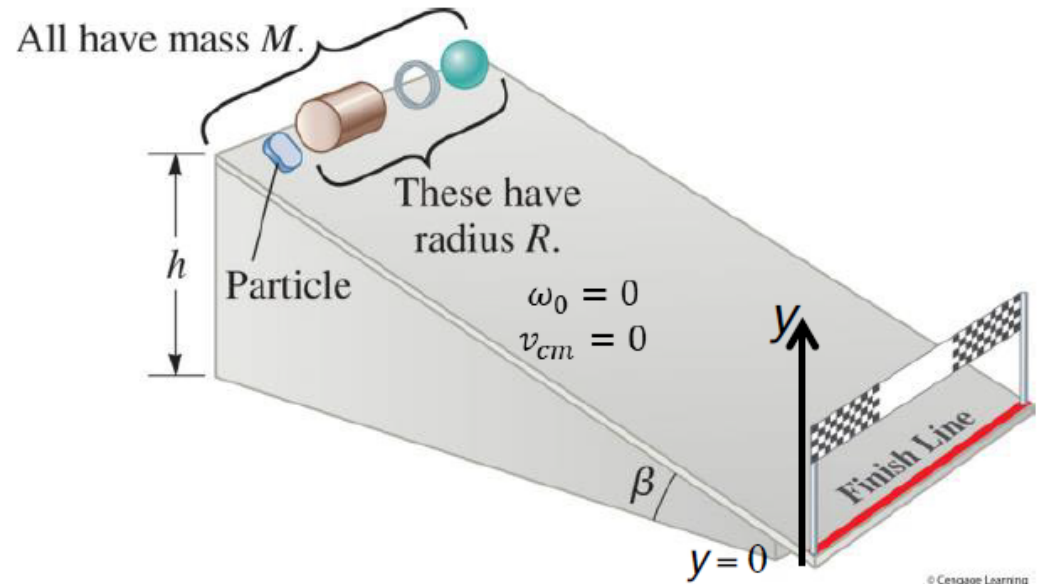
Downhill race between four objects

Hoop: $I = MR^2$

Cylinder: $I = \frac{1}{2}MR^2$

Sphere: $I = \frac{2}{5}MR^2$

Skier (particle)



The winner is:

Caveat: No
energy losses
i.e. Frictionless
for the skier

- A. Hoop
- B. Cylinder
- C. Sphere
- D. Skier

The Race

Downhill race between four objects

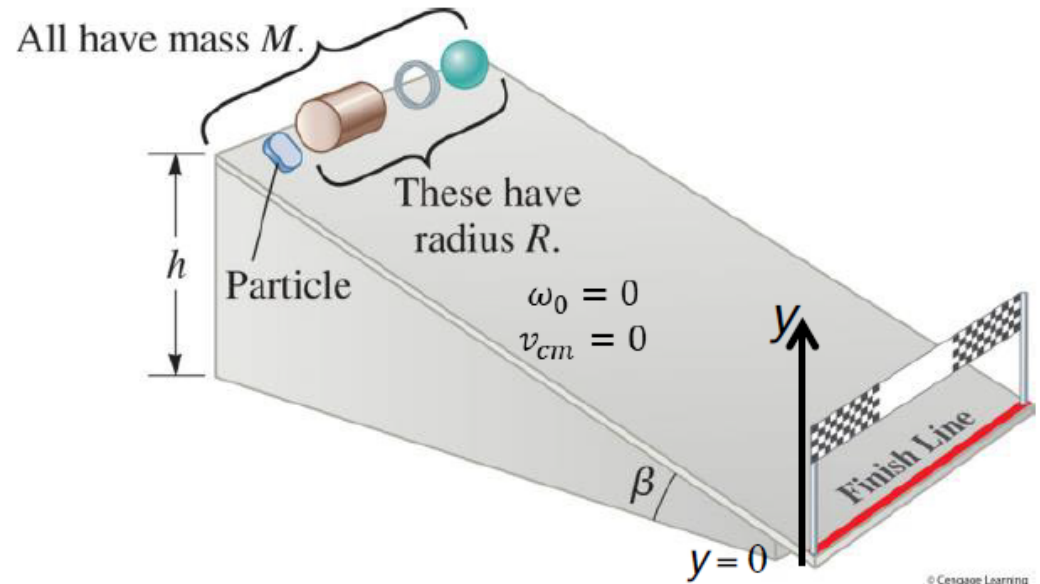
Hoop: $I = MR^2$

Cylinder: $I = \frac{1}{2}MR^2$

Sphere: $I = \frac{2}{5}MR^2$

Skier (particle)

We can use our 2D cylinder model to test all three round cases just by changing the moment of inertia I



The winner is:

- A. Hoop
- B. Cylinder
- C. Sphere
- D. Skier

The Race.....

$$E_{top} = E_{bottom} \quad (\text{cons. of energy})$$

For the rolling objects:

$$U_{gi} = K_{CM} + K_r$$

$$Mgh = \frac{1}{2}Mv_f^2 + \frac{1}{2}I_{CM}\omega^2$$

$$I_{CM} = cMR^2 \quad \omega = \frac{v_f}{R}$$

$$Mgh = \frac{1}{2}Mv_f^2 + \frac{1}{2}cMR^2 \frac{v_f^2}{R^2}$$

$$v_f = \sqrt{\frac{2gh}{1+c}}$$

For the particle:

$$U_{gi} = K_{CM}$$

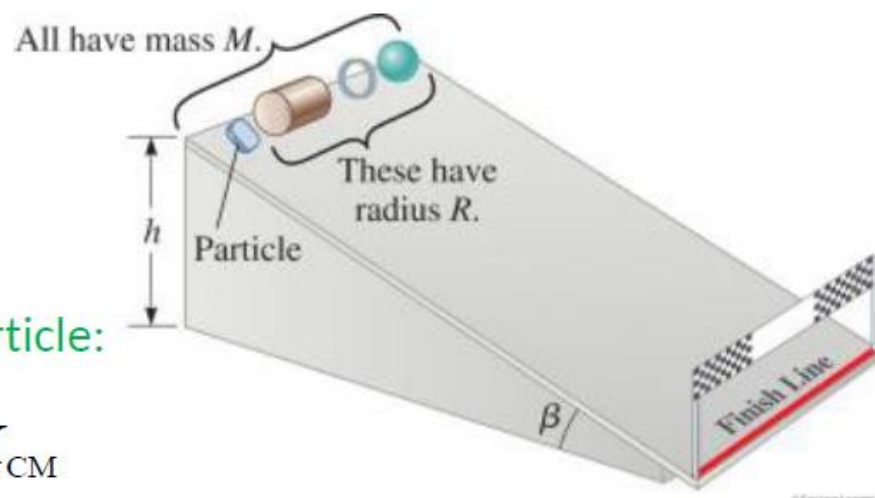
$$Mgh = \frac{1}{2}Mv_f^2$$

particle

cylinder $c = \frac{1}{2}$

hoop $c = 1$

sphere $c = \frac{2}{5}$



$$v = \sqrt{2gh} \quad \text{1}$$

$$v = \sqrt{\frac{4gh}{3}} \quad \text{3}$$

$$v = \sqrt{gh} \quad \text{4}$$

$$v = \sqrt{\frac{10gh}{7}} \quad \text{2}$$

Doesn't depend on M or R , just on c (the shape – smallest I)

The Race.....

$$E_{top} = E_{bottom} \quad (\text{cons. of energy})$$

For the rolling objects:

$$U_{gi} = K_{CM} + K_r$$

$$Mgh = \frac{1}{2} Mv_f^2 + \frac{1}{2} I_{CM} \omega^2$$

$$I_{CM} = cMR^2 \quad \omega = \frac{v_f}{R}$$

$$Mgh = \frac{1}{2} Mv_f^2 + \frac{1}{2} cMR^2 \frac{v_f^2}{R^2}$$

$$v_f = \sqrt{\frac{2gh}{1+c}}$$

For the particle:

$$U_{gi} = K_{CM}$$

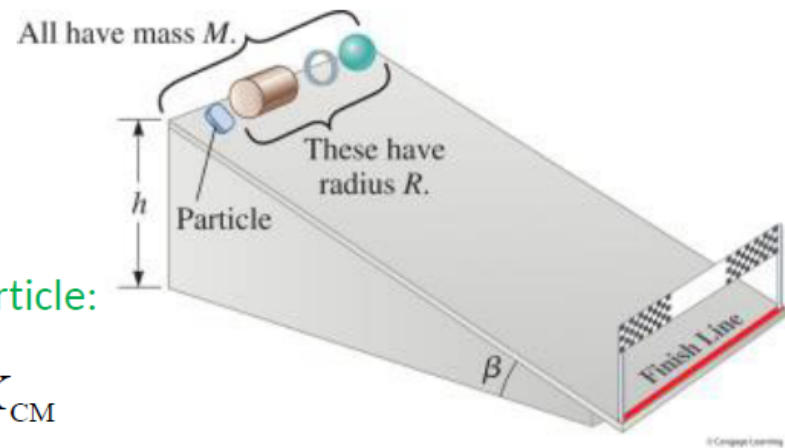
$$Mgh = \frac{1}{2} Mv_f^2$$

particle

cylinder $c = \frac{1}{2}$

hoop $c = 1$

sphere $c = \frac{2}{5}$



$$v = \sqrt{2gh} \quad \text{1}$$

$$v = \sqrt{\frac{4gh}{3}} \quad \text{3}$$

$$v = \sqrt{gh} \quad \text{4}$$

$$v = \sqrt{\frac{10gh}{7}} \quad \text{2}$$

- Challenging for game engine -- requires really good energy conservation to get these results!